
DTU

DTU

Multi-objective Optimization

Lecture 10

Course evaluation

It is possible to evaluate the course anonymously on Campusnet. **PLEASE EVALUATE THE COURSE**. The evaluation is open today Tuesday 20/1 and until Thursday 22/1, 23.59. We actually use these evaluations to improve the course.

Multi Objective optimization, why ?

All (almost) real-world optimization problems are multi-objective. But in the large majority of cases, are solved/optimized a single-objective models, because multi-objective optimization is too hard.

This course is about mathematical programming **modeling**. Hence we do not expect you to understand the details behind the optimization algorithms, but you need an intuitive understanding. We give references to the research behind the algorithms, for further self-study, but this is not part of the course.

Example: TA work schedule I

The plan for this course was made by an extended model of the exercise in the material.

Example: TA work schedule II

The objectives:

- Minimize no of work days
- Maximize reward (when TA's want to work) (or Minimize the negative reward)
- Minimize penalty (when TA's don't want to work)
- Minimize no of gap timeslots

Example: TA work schedule III

The constraints:

- Ensure the demand is satisfied
- Ensure that each TA work the correct no of hours
- Ensure that when TA has a work day, at least 3 hours of work is allocated
- plus a few other small details

Example: TA work schedule IIII

This year I solved it Multi-objectively (4 objectives, Minimize):

- The Kirklin-Sayin algorithm was used
- It took app 6600 sec.
- It found 1734 optimal solutions !

Example: TA work schedule IIIII

```
Res: 1734
No Pareto optimal points: 1734
1 Pen: 4.0 Rew: 290.0 WorkDays: 48.0 ConS: 8.0
2 Pen: 4.0 Rew: 289.0 WorkDays: 48.0 ConS: 7.0
3 Pen: 4.0 Rew: 283.0 WorkDays: 48.0 ConS: 6.0
....
1730 Pen: 97.0 Rew: 299.0 WorkDays: 41.0 ConS: 1.0
1731 Pen: 97.0 Rew: 299.0 WorkDays: 43.0 ConS: 0.0
1732 Pen: 98.0 Rew: 302.0 WorkDays: 42.0 ConS: 1.0
1733 Pen: 99.0 Rew: 301.0 WorkDays: 41.0 ConS: 1.0
1734 Pen: 100.0 Rew: 303.0 WorkDays: 44.0 ConS: 0.0
Successfull end  time: 6679.08
```

Multi Objective optimization, algorithms and theory

Multi-objective optimization is an important topic, but despite research in mathematical optimization methods, these solution methods are still much weaker than single-objective methods. In March 2023, multi-objective modeling became possible in Julia/JuMP, hence multi-objective modeling became much easier. We believe this is the first time this has become directly accessible in a Mathematical Programming modeling language/package. Notice that the implementation part of multi-objective optimization is easy, but the theory behind multi-objective optimization is deep.

Multi Objective modelling

The direct model extension is really simple:

$$
\begin{aligned}
&\texttt{Maximize/Minimize} && \textstyle\sum_i F_i^1 \cdot x_i \\
& && \textstyle\sum_i F_i^2 \cdot x_i \\
& && \dots \\
& && \textstyle\sum_i F_i^k \cdot x_i \\
&\texttt{Subject to :} && \\
& && \textstyle\sum_i A_{i,j} \cdot x_i \quad \leq \quad B_j \; \forall \; j \\
& && x_i \in R/Z/\{0,1\}
\end{aligned}
$$

The **only** difference is the addition of one or more objective functions. This simple extension creates a whole new class of optimization problems.

Multi Objective optimization

Given the model from the previous slide, how can it be optimized ? As it turns out, this is complex and is an active research field in Thomas is actively contributing to the state-of-art. There are two different approaches:

- Criteria space algorithms: Apply a number of sequential single-objective (MIP) optimizations to the problem.
- Decision space approaches: Customize Branch & Cut algorithms to multiple-objectives.

All the algorithms available in Julia/JuMP are of the Criteria space type. Hence, a single objective MIP solver (HiGHS or Gurobi) is a sub-routine of the approach.

What is multi-objective ordering ?

Given two different feasible solutions, lets call them x_1 and x_2. If we assume to have 2 objective functions, the objective vector of each solution is: $F(x_1) = (F^1(x_1), F^2(x_1))$ and $F(x_2) = (F^1(x_2), F^2(x_2))$. We say that a solution (x_1) **dominates** another solution (x_2) when (assuming minimization) if: $F^1(x_1) < F^1(x_2)$ and $F^2(x_1) \leq F^2(x_2)$ **or** $F^1(x_1) \leq F^1(x_2)$ and $F^2(x_1) < F^2(x_2)$

In words: If one solution is definitely better on one objective and at least as good on the other, then this solution **dominates** this solution.

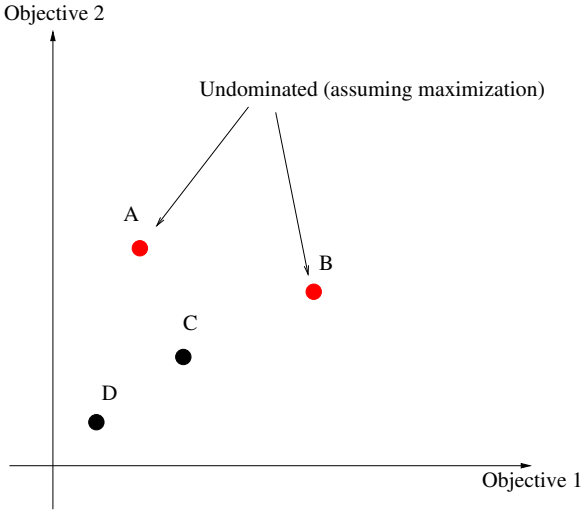
Bi-Objective modelling

In this course we will **only** consider bi-objective models, i.e. models with 2 objective functions:

$$
\begin{aligned}
&\texttt{Maximize/Minimize} && \textstyle\sum_i F_i^1 \cdot x_i \\
& && \textstyle\sum_i F_i^2 \cdot x_i \\
&\texttt{Subject to :} && \\
& && \textstyle\sum_i A_{i,j} \cdot x_i \quad \leq \quad B_j \; \forall \, j \\
& && x_i \in R/Z/\{0,1\}
\end{aligned}
$$

This has the advantage that we can show the figure in two-dimensional space. We assume maximization of both objectives in the examples.

Dominance visually



Pareto optimal solutions

We define the set of **Pareto optimal solutions**: The set of solutions which are not dominated by any other solution. The point is that if the objective functions are our only measure, then the optimal solution for a multi objective model, is the Pareto optimal set, i.e. a **set** of solutions.

In two dimensions, we can easily illustrate different types of Pareto fronts.

Continuous Pareto front (MIP)

If our model is a Bi-objective Linear Programming (Bi-MOLP) model (with continuous variables in both objective functions):

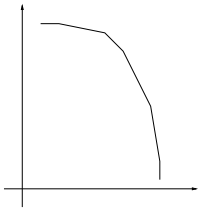


Figure: A continuous Pareto Front

Point-wise Pareto front

If our model is a Bi-objective Mixed Integer Programming (Bi-MOMIP), where there in at least one of the objective functions is only integer variables in at least one of the objective functions:

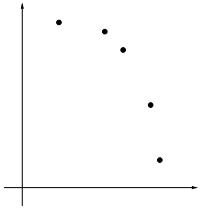


Figure: A point-wise Pareto Front

In this course we only consider point-wise Pareto front problems.

Integer point point-wise Pareto front

If our model is a Bi-objective Mixed Integer Programming (Bi-MOMIP), where both objective functions only contains integer variables **and** each objective only contains integer objective coefficients:

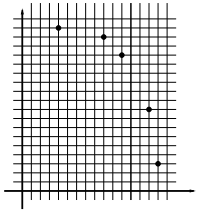


Figure: A point-wise Pareto Front in an integer grid

Lexiographic optimization, Upper Left

If we first maximize according to one objective, add a constraint to forbid a decline in this objective, and then optimize according to the other objective, we get a **lexicographic** solution either Upper-Left or Lower-Right.

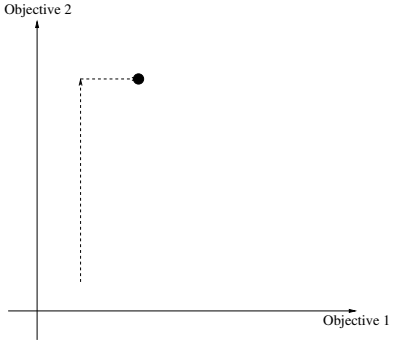


Figure: Finding Upper-Left (UL) Pareto point through lexiographic z2 $\rightarrow$ z1 optimization

Lexiographic optimization, Lower Right

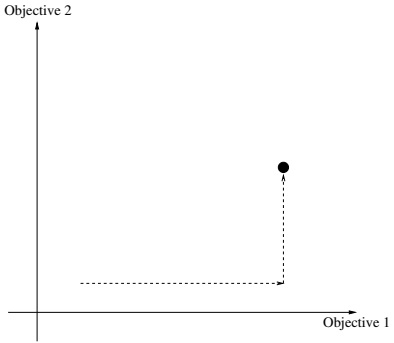
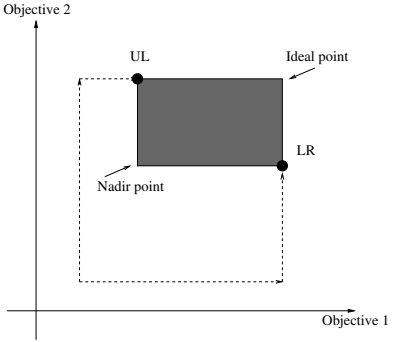


Figure: Finding Lower-Right (LR) Pareto point through lexiographic z1 $\rightarrow$ z2 optimization

The two-dimensional criteria space



Multi Objective optimization algorithms

This is an active research area, and the algorithms offered in JuMP are:

- ϵ-Constraint algorithm, classic algorithm. Works only for two objectives
- Dichotomy. (**Don't use**) Notice that this method does **not** find the full pareto front, only the extreme points of the pareto front, i.e. the solutions which can be found using different secularization's of the objectives.
- Chalmet (1986), (**Don't use**)
- DominguezRios (2021)
- KirlikSayin (2014)
- TambyVanderpooten (2021)

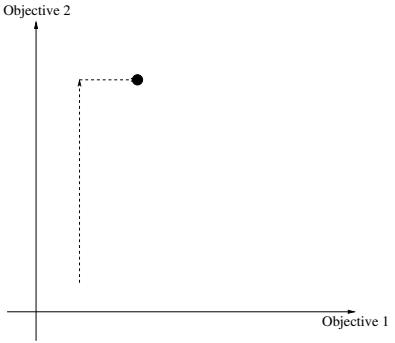
In this course we will only use the ϵ-Constraint algorithm.

The ϵ-Constraint algorithm

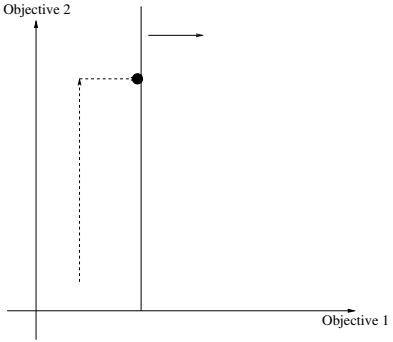
Algorithm 1: The ϵ-Constraint algorithm

```
1 solution-exists=true;
2 while solution-exists do
3     (solution-exists,z1,z2)=LexioGraphicZ2Z1();
4     if solution-exists then
5         AddParetoPoint( (z1,z2) );
6         SetLowerBoundZ1(z1+δ);
```

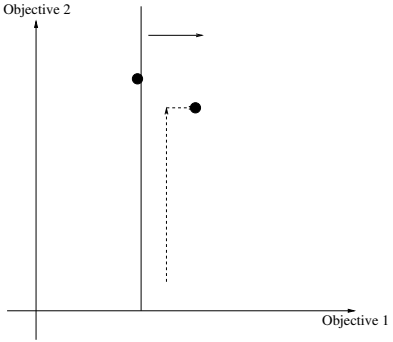
Algorithm start



Algorithm add constraint



Algorithm repeat lexiographic



When does it end ?

When the lexiographic optimization cannot find a feasible solution.

How to do it in practice

Because of the new development, doing Multi-Objective modelling in Julia/JuMP is relatively simple, however:

- We can only either maximize or minimize both functions. Hence we may have to multply by -1
- Some of the Multi-Objective optimization algorithms newly implemented an possibly a little "beta"

Multi-objective Julia/JuMP in practice

```
#************************************************************************
# Multi-Objective Incredible Chairs 4
using JuMP
using HiGHS
using MultiObjectiveAlgorithms
#************************************************************************
```

Multi-objective Julia/JuMP in practice

```
# Model
MO_IC = Model()
@variable(MO_IC, x[1:C]>=0, Int)
# Objective 1: Maximize Profit
@expression(MO_IC, Profit_expr, sum(Profit[c] * x[c] for c=1:C))
# Objective 2: Maximize Rating
@expression(MO_IC, Rating_expr, sum(Rating[c] * x[c] for c=1:C))
# Define combined BI-OBJECTIVE
@objective(MO_IC, Max, [Profit_expr, Rating_expr])

# Capacity limitations
@constraint(MO_IC, [p=1:P], sum( RecourseUsage[p,c] * x[c] for c=1:C)
<= Capacity[p])
```

Multi-objective Julia/JuMP in practice

```
#**************************************************************************
# Solve

# Set MIP solver
set_optimizer(MO_IC, () ->
   MultiObjectiveAlgorithms.Optimizer(HiGHS.Optimizer))
set_silent(MO_IC)

# Set MO solver
set_attribute(MO_IC, MultiObjectiveAlgorithms.Algorithm(),
              MultiObjectiveAlgorithms.EpsilonConstraint())

optimize!(MO_IC)
#solution_summary(MO_IC)
#**************************************************************************
```

Multi-objective Julia/JuMP in practice

```
#**************************************************************************
# Solution
if result_count(MO_IC)>=1
    println("No Pareto optimal points: ",result_count(MO_IC))
    for i in 1:result_count(MO_IC)
        y = objective_value(MO_IC; result = i)
        println("O1: ",round.(Int,y[1]), " O2: ",round.(Int,y[2]))
    end

else
    println("No solutions found")
end
#**************************************************************************
```

OR is fantastic: How can I study more ???

- 42101 (BSc): Introduction to Operations Research
- 42586 (BSc): Decisions under uncertainty
- 42112 (MSc): Mathematical Programming Modelling
- 42114 (MSc): Integer Programming
- 42115 (MSc): Network Optimization
- 42136 (MSc): Large Scale Optimization using Decomposition
- 42137 (MSc): Optimization using Metaheuristics
- 42142 (MSc/PhD): Recent Research Results in Management Science

Final exam, time and place

The final exam is a 3 hour written time and place:

- 23. januar 2026, 09:00 - 12:00, 358/925, Campus Lyngby
- 23. januar 2026, 09:00 - 12:00, 358/060b, Campus Lyngby
- 23. januar 2026, 09:00 - 12:00, 358/060a, Campus Lyngby
- 23. januar 2026, 09:00 - 12:45, 358/914, Campus Lyngby

How is the exam

The exam consists of 2 parts:

- Multiple-choice part, (60 % weight).
- Modelling and implementation part, formulate a mathematical programming model, implement the model in Julia and solve it, (40 % weight). It is one problem with 3 sub questions. For each question a julia program should be handed in.

The three parts will not be inter-related and can be done in any order. A good strategy is probably to not get stuck in a problem, but use the time on the parts which seems most accessible.

What to hand in

As a result of the exam, you should hand in:

- Answers to the multiple choice test
- Julia/JuMP program/model, which can be executed, i.e. as .jl file, one file for each sub-question

Final exam

The best preparation for the exam: Make sure you have done and understood all the exercises of the course.

Notice: We will **not** pose any math-heuristic algorithm questions in the exam.